

AI Python Engineering Assignment

Objective

Build a simple Retrieval-Augmented Generation (RAG) application that can answer questions from a set of PDF documents.

We are **not** evaluating how many frameworks or libraries you use. We are evaluating your ability to read documentation, integrate multiple components, and build a clean, working solution.

Time Limit

Please spend **no more than 3 hours** on this assignment.

Technology Requirements

Language

- Python 3.11+

LLM

- Use **OpenRouter** (<https://openrouter.ai/>)
- Use **free models only**

Vector Database

- Use **Qdrant** (Local Docker or Qdrant Cloud Free Tier) <https://qdrant.tech/>

AI Tools

You may use any AI coding assistants including Cursor, Claude Code, Codex, ChatGPT, Gemini, etc.

Using AI tools is encouraged.

Problem Statement

You will be provided with one or more PDF documents.

https://drive.google.com/drive/folders/1VN7E4YfANr4Zde4sias_JZw6piyLHvu2?usp=sharing

Your task is to build a Python application that can answer questions using only the information contained in those PDFs.

Functional Requirements

Your application should:

1. Parse all provided PDF documents.
 2. Generate embeddings for the document content.
 3. Store the embeddings in Qdrant.
 4. Retrieve relevant content for a user query.
 5. Generate an answer using an OpenRouter free model.
 6. Return accurate citations with every answer.
-

Citation Requirements

Every answer **must** include:

- Document name
- Page number
- The retrieved text snippet used to generate the answer

Example:

Question:

What is the leave policy?

Answer:

Employees are entitled to 24 annual leave days.

Source:

employee_handbook.pdf

Page 17

Retrieved Text:

"All full-time employees receive 24 days of annual leave..."

Unknown Questions

If the answer cannot be found in the provided documents, your application should clearly respond that the information is not available in the supplied documents.

Do not fabricate answers.

Deliverables

1. GitHub Repository

Include complete source code.

2. README (Maximum 1 Page)

Include:

- Overall architecture
- Libraries used
- Embedding model used
- Any assumptions made
- How to run the application

Keep the README concise.

3. Demo Video (1–2 Minutes)

Record a short screen recording demonstrating:

- Running the application from your local machine.
- Asking at least **three questions** from the supplied PDFs.
- The generated answers.
- The corresponding citations (document name and page number) for each answer.
- One question whose answer is **not** present in the PDFs, showing that your application correctly responds that the information is unavailable.

The video does not need narration, but you may briefly explain your implementation if you wish.

Evaluation Criteria

We will evaluate your submission based on:

1. Functional Correctness (40%)

- Successfully processes all supplied PDFs
- Correctly retrieves relevant information
- Produces useful answers

2. Citation Accuracy (30%)

Every answer should correctly reference:

- Document name
- Page number
- Retrieved text

The cited text must support the generated answer.

3. Code Quality (20%)

- Clean Python code
- Good project structure

- Readable implementation
- Appropriate error handling

4. Documentation (10%)

- Clear README
 - Easy to run
-

Important Notes

- AI coding assistants are allowed and encouraged.
- There is no single correct implementation.
- Keep the implementation simple.
- We value correctness, readability, and engineering quality over feature richness.

We will evaluate your application using our own set of questions based on the supplied PDFs. Your application should answer only from the provided documents and include accurate citations for every response.